Amazon Elastic Kubernetes Service

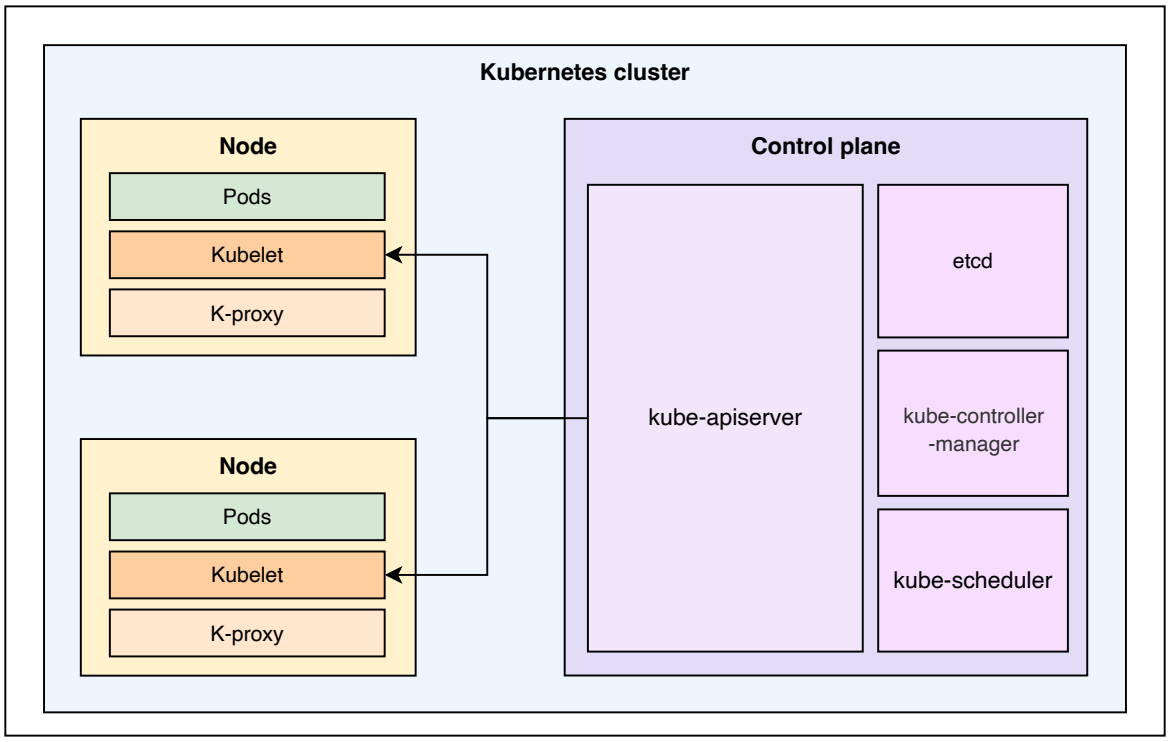
Explore Amazon Elastic Kubernetes Service (EKS) to understand Kubernetes cluster architecture, deployment, and management on AWS. Learn how EKS automates container orchestration with secure access control, observability tools, and scalable deployment options. Gain insights into EKS Distro and EKS Anywhere for hybrid and on-premises Kubernetes environments.

Kubernetes (K8s) is an open-source and extensible container orchestration platform. It automates the deployment, scaling, and management of containerized applications. Kubernetes allows the management and coordination of clusters of containers across multiple hosts, providing services such as fault tolerance and scalability. In this lesson, we will review K8 and its major components and understand how EKS and EKS Distro work.

K8s components

Kubernetes orchestrates containerized applications within a Kubernetes cluster, which consists of a control plane and worker nodes. Each cluster has a pool of worker nodes to run the containerized applications on Pods, which represent one or more co-located containers. The control plane is used to make decisions about the cluster, schedule nodes, and manage them.

Ask



K8s components

The cluster contains multiple worker nodes in a production environment, and the control plane runs across multiple machines, ensuring high availability and fault tolerance.

Control plane components

The main components of the control plane are discussed below:

- **etcd:** This is the key-value storage for storing the Kubernetes cluster's data, service discovery details, and API objects.
- **kube-scheduler:** It schedules newly created Pods on a worker node.
- **kube-controller-manager:** It runs controller processes such as the node controller for handling node failures and the job controller. There is a separate controller component for cloud integration.
- **kube-apiserver:** The Kubernetes API server is the primary managem entity for the cluster, receiving all REST requests.

Ask

Node components

Every worker node in the Kubernetes cluster also runs some components, as shown in the illustration above. Kubernetes is designed to be highly flexible, supporting a variety of container runtimes such as containerd, CRI-O, and even Docker Engine through its Container Runtime Interface (CRI). A high-level overview of node components is as follows:

- **Pods:** A Pod is the smallest deployable unit in Kubernetes and represents one or more co-located containers.
- **Kubelet:** An agent within the nodes that manages the containers in the Pod, ensuring that they are running and healthy.
- **Kube-proxy:** It allows network communication to the Pods from the internet or inside the cluster.

Operating and securing a Kubernetes control plane across production environments requires deep expertise in cluster provisioning, scalability, patching, network configuration, and integration with cloud services. This makes self-managed Kubernetes operationally burdensome for most teams.

Amazon Elastic Kubernetes Service

Amazon Elastic Kubernetes Service (EKS) is a fully managed service that simplifies managing Kubernetes clusters on AWS. EKS simplifies containerized applications' deployment, management, and scaling using Kubernetes on AWS infrastructure. EKS supports different compute services, from serverless Fargate to EC2, providing high availability, strong security, and seamless integration with other AWS services to improve development and operational efficiency.

Ask

EKS is versatile and adaptable to various use cases, offering developers and organizations the flexibility, scalability, and efficiency required to modernize their applications. It efficiently scales resources to handle fluctuating traffic loads while maintaining high availability and performance.

How EKS works

An EKS cluster consists of two major components: the Control plane and the Nodes. When provisioning an EKS cluster to run an application, the control plane is hosted on the AWS-managed VPC, whereas the worker nodes and other infrastructure are hosted in the customer-managed VPC. The worker nodes may communicate with the control plane through the managed API endpoint in the control plane. The worker nodes are connected to the control plane through the API or an EKS-managed ENI, placed in the customer subnets.

EKS workflow

Ask

Example: Hosting a web application

To host a highly available web application on EKS, we begin by provisioning a cluster with a managed control plane and a pool of worker nodes in our own VPC. An Application Load Balancer (ALB) is typically configured to receive user traffic and route it to the appropriate Pods running on the nodes. The control plane ensures that the desired state of the application is maintained, orchestrating deployments, scheduling Pods, and performing health checks.

A crucial organizational layer in this setup is the namespace. **Kubernetes namespaces** allow us to isolate and group related resources, such as services, Pods, and ConfigMaps. For instance, we may deploy our web application in a dedicated "production" namespace while keeping our staging and development workloads separate. This makes it easier to apply resource quotas, access policies, and clean up procedures in a targeted way.

Applications are defined and deployed using manifests, which are YAML or JSON configuration files that describe the desired state of Kubernetes objects, such as deployments, services, and ingress controllers. These manifests specify the container image to use, environment variables, replicas, and other configuration metadata. They act as a declarative source of truth for the application infrastructure.

To apply these manifests and interact with the cluster, developers use `kubectl`, the command-line interface for Kubernetes. With `kubectl`, we can deploy resources using `kubectl apply -f`, inspect running workloads using `kubectl get pods`, view logs using `kubectl logs`, and troubleshoot issues using `kubectl describe`. It serves as the primary operational tool for controlling and observing the state of workloads in the EKS environment.

This combined approach underpins a secure and manageable EKS deployment. It ensures that the web application can scale, recover, and evolve while mair a clear operational model.

Ask

Secure access and permissions in EKS

Security in EKS is governed by a two-tiered model: IAM for AWS resource-level access and Kubernetes-native RBAC for in-cluster operations. Combining these models ensures that both cloud-native and Kubernetes-native access controls are respected.

IAM roles for service accounts (IRSA)

IRSA allows fine-grained IAM permissions to be applied to individual Pods through Kubernetes service accounts. This mechanism leverages the cluster's OIDC identity provider to securely assume IAM roles without the need for long-lived credentials inside containers. This involves the following steps:

1. Enable the OIDC provider for the EKS cluster using the AWS Management Console or CLI.
2. Create an IAM role with a trust policy that references the EKS OIDC provider and a specific service account.
3. Annotate the Kubernetes service account with the IAM role ARN using metadata annotations.

This setup enables applications running in Pods to interact with services like Amazon S3, DynamoDB, or Secrets Manager securely. The illustration below shows how roles are associated with the Pods running in EKS:

Ask

Assigning roles to EKS nodes for limited access

Common use cases for IRSA include allowing a logging container to write to CloudWatch and letting a machine learning workload pull training data from an S3 bucket.

Kubernetes RBAC

Kubernetes role-based access control (RBAC) is a fundamental mechanism for defining and enforcing fine-grained permissions within an EKS cluster. Unlike IAM, which governs access to AWS resources, RBAC focuses on in-cluster permissions: who can access what, and with what level of control. It is particularly useful in environments with multiple teams, automated pipelines, or diverse service accounts that require scoped access to specific namespaces or resources.

Ask

RBAC policies are defined using Kubernetes-native resources: `Role` and `ClusterRole` to define sets of permissions, and `RoleBinding` and

`ClusterRoleBinding` to associate those permissions with users, groups, or service accounts. `Roles` are namespace-scoped, while `ClusterRoles` apply across the entire cluster. This makes it easy to delegate responsibilities and isolate access boundaries.

To implement RBAC in an EKS cluster:

1. Define a role or ClusterRole that includes a list of API groups, resource types, and allowed actions such as `get`, `list`, `create`, and `delete`.
2. Bind that role to a user or service account using a `RoleBinding`, for namespace-limited access, or a `ClusterRoleBinding`, for cluster-wide access.
3. Apply these manifests using `kubectl apply -f`, and test the permissions using impersonation or by attempting different operations under the bound identity.

For example, a development team may be given read-only access to application logs and resource configurations within a staging namespace, while a CI/CD pipeline has administrative privileges to deploy and update workloads in production. This ensures that every principal, human or machine, operates with the least privilege necessary.

When used in combination with IAM roles for service accounts (IRSA), RBAC provides a comprehensive security model that controls access both inside and outside the cluster. This layered approach is essential for building secure, compliant, and maintainable EKS environments.

> **Best practice:** Use RBAC to scope permissions narrowly and combine with IRSA for external AWS access.

Ask

Observability and monitoring

Observability in EKS involves using AWS-native services in combination with Kubernetes tools to ensure a detailed understanding of system and application behavior. It spans metrics, logs, and traces, helping developers and operators monitor performance, detect anomalies, and maintain reliable service delivery.

Amazon CloudWatch Container Insights offers real-time access to metrics like CPU usage, memory consumption, disk I/O, and the status of container lifecycles. By deploying the CloudWatch agent as a DaemonSet with appropriate permissions via IRSA, EKS clusters can automatically push these metrics into the CloudWatch console, where they can be visualized and used to trigger alarms or dashboards. These insights help detect trends such as gradual resource exhaustion or sudden Pod failures, facilitating proactive maintenance.

For custom application metrics that do not fall under standard resource monitoring, CloudWatch embedded metric format (EMF) allows developers to embed structured logs that the CloudWatch agent can parse into usable metrics. This allows teams to monitor business-relevant KPIs or internal application behavior and to trigger alarms or scaling actions based on them.

In environments composed of distributed microservices, AWS X-Ray provides visibility into request traces as they traverse services. Applications instrumented with the X-Ray SDK can send segment data to the X-Ray DaemonSet running in the cluster. This data is crucial for identifying service bottlenecks, debuggin latency issues, and ensuring end-to-end transaction traceability.

Ask

Together, these tools offer a cohesive observability stack that is well-integrated with the EKS environment and AWS ecosystem, empowering teams to maintain operational excellence across diverse deployment scenarios.

Common failure modes

When running Kubernetes workloads in Amazon EKS, several operational issues appear consistently. These failures can often be identified using standard Kubernetes commands, while AWS-native observability tools such as CloudWatch Container Insights, CloudWatch Logs, and AWS X-Ray provide enhanced visibility into their root causes. Below are some of the most commonly observed problems and how they can be approached:

- **CrashLoopBackOff or ImagePullBackOff:** These errors indicate that containers are either repeatedly crashing or failing to start due to image retrieval issues. Common causes include misconfigured container commands, missing environment variables, or authentication failures with the image registry.

  We can use the following command to inspect the event timeline and container states:

  ```
  kubectl describe pod <pod-name>
  ```

  To identify the root cause from recent output, we can use the following command:

  ```
  kubectl logs <pod-name>
  ```

  We can use CloudWatch Container Insights to identify resource constraints like memory exhaustion, that may be contributing to the failures.

  We can use CloudWatch Logs to capture the application's standard error output, which often provides the clearest explanation for recurring crashes.

Ask

- **Event stream anomalies:** These involve failures that do not manifest as container errors but occur at the cluster level, such as failed Pod scheduling, denied webhook requests, or unsatisfied resource constraints.

  We can use the following command to review the cluster's recent activity, such as failed scheduling attempts or rejected webhook operations.

  ```
  kubectl get events --sort-by=.metadata.creationTimestamp
  ```

  We can use Container Insights to identify pending Pod counts, node availability, and scheduler activity to help contextualize failures.

  We can configure CloudWatch Logs to capture events from the scheduler or admission webhooks for detailed analysis.

- **HPA behavior anomalies:** A Horizontal Pod Autoscaler may not function properly if the metrics server (such as the CloudWatch adapter) isn't installed or configured correctly. It's important to verify metric flow and reassess scaling thresholds, cooldown settings, and utilization targets. We can also use Container Insights to validate CPU and memory metrics that the HPA uses for scaling decisions. We can also use CloudWatch EMF to emit custom application metrics if the HPA is configured to scale based on business logic rather than standard resource usage.

- **Network issues:** Networking misconfigurations can disrupt service discovery or inter-Pod communication. To effectively diagnose DNS resolution issues within Pods, we can use the following command to review CoreDNS logs for clues and simultaneously validate that NetworkPolicies and associated security group rules are not unintentionally blocking traffic:

  ```
  kubectl exec <pod-name> -- nslookup <service-name>
  ```

  Container Insights and VPC Flow Logs can provide visibility into dropped packets or denied traffic due to misconfigured security groups or network policies.

Ask

By combining standard Kubernetes tooling with AWS observability services, teams can efficiently identify, trace, and resolve complex runtime issues in EKS clusters. This integrated approach ensures higher reliability, faster recovery, and better operational visibility in production environments.

EKS Distro

Amazon EKS Distro is an open-source distribution of Kubernetes provided by AWS. It offers the same core components that power Amazon EKS for creating and managing containerized applications. The difference between EKS and EKS Distro is that the latter can be installed and managed locally. EKS Distro can be used on-premises, in the cloud, or in any system. It provides a path to having essentially the same Amazon EKS Kubernetes distribution running wherever you need to run it.

EKS Distro automatically creates and manages Kubernetes clusters. It offers a consistent Kubernetes experience across AWS cloud and data centers. It alleviates the need to manually track, update, and determine Kubernetes compatibility across different teams. It also offers installable, reproducible Kubernetes builds for cluster creation as well as security patches even after the community support expires.

Amazon EKS Anywhere

EKS Anywhere is customer-managed container management software bui
AWS to run and manage Kubernetes clusters on-premises or at the edge. It

Ask

utilizes EKS Distro to offer the same reliable, consistent service as AWS EKS on-premises.

EKS Anywhere allows organizations to leverage the managed Kubernetes service model provided by EKS while running Kubernetes clusters on their infrastructure, providing flexibility and control over the workloads deployed. Using familiar AWS

Ask